



Virtualización y Consolidación de Servidores

Trabajo Final Integrador

“Caso de estudio: Blog Personal:”

Fecha de presentación: 28/06

- **Docente:** Carriles, Luis María
- **Alumno:** Miranda López, Victor Manuel
- **Carrera:** Ingeniería en Sistemas de Información
- **Comisión:** 5K4

Año 2026

Tabla de contenidos

1. Introducción y objetivos	3
1.1. Especificaciones a cumplir	3
2. Descripción de la infraestructura	3
2.1. Entorno de virtualización	4
2.2. Esquema de direccionamiento IP	4
3. Aprovisionamiento de los contenedores	4
4. Configuración de la capa de datos (Contenedor DB)	5
4.1. Instalación del motor	5
4.2. Ajuste de bajo consumo de memoria	5
4.3. Creación de la base, usuario y esquema	6
5. Configuración de la capa de aplicación (Contenedor Blog).....	6
5.1. Arquitectura de la solución	6
5.2. Componentes instalados	6
5.3. Servicio systemd	6
5.4. Configuración de Nginx	7
6. Funcionalidad del blog	7
6.1. Características implementadas	7
6.2. Consideraciones de seguridad	7
7. Publicación del blog bajo la URL institucional	9
7.1. Adaptación del servidor web interno (Nginx)	9
7.2. Adaptación de la aplicación (Flask)	9
7.3. Dificultad encontrada: entrega de recursos estáticos	9
7.4. Solución adoptada	10
8. Pruebas y verificación	10
8.1. Conectividad de la aplicación	10
8.2. Comunicación entre contenedores	10
8.3. Prueba funcional	11
9. Conclusiones	11
Anexo: stack tecnológico	12

1. Introducción y objetivos

El presente informe documenta el desarrollo e implementación del Trabajo Práctico Final de la cátedra Virtualización y Consolidación de Servidores. El objetivo consiste en implementar un servicio de Blog Personal desplegado sobre una infraestructura virtualizada basada en Proxmox VE, respetando las especificaciones de recursos, direccionamiento de red y nomenclatura indicadas en la consigna.

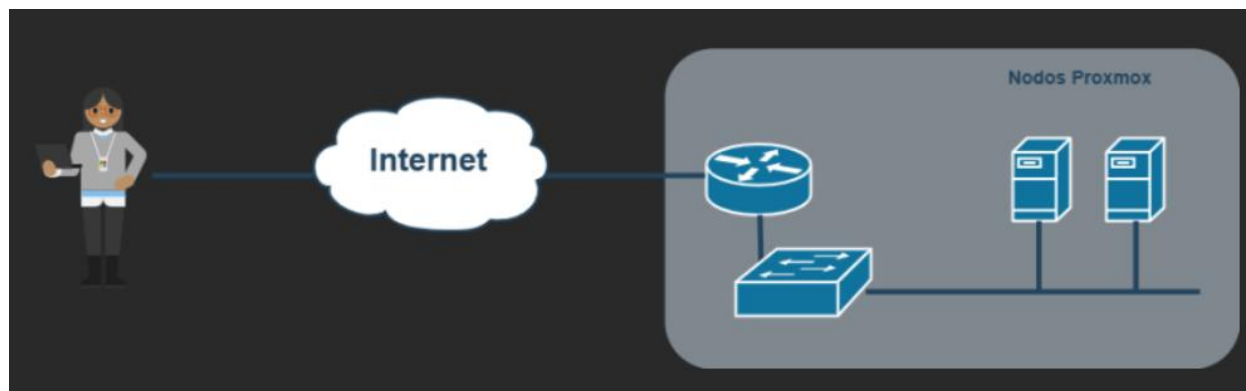
La solución aplica el concepto de consolidación de servidores mediante la separación de responsabilidades en dos contenedores LXC independientes: una capa de aplicación web y una capa de datos. Ambos contenedores conviven sobre el mismo nodo físico, demostrando el aprovechamiento eficiente del hardware a través de la virtualización liviana.

1.1. Especificaciones a cumplir

- Datos personales del alumno publicados en el blog.
- Imagen personal del alumno.
- Posibilidad de actualizar la página principal agregando una nueva entrada.
- Informe del desarrollo e implementación disponible en formato PDF y publicado en la propia implementación.
- Despliegue sobre dos contenedores con recursos acotados (128 MB de memoria, 1 procesador, 8 GB de almacenamiento e IP estática cada uno).

2. Descripción de la infraestructura

La topología cátedra consiste en un acceso vía Internet hacia los nodos Proxmox a través de la dirección `nap.frt.utn.edu.ar`. Sobre ese entorno se aprovisionaron los dos contenedores requeridos, conectados a la red interna 172.16.90.0/24.



2.1. Entorno de virtualización

Plataforma	Proxmox Virtual Environment 8.2.2
Tipo de virtualización	Contenedores LXC (virtualización a nivel de SO)
Nodo	pve
Acceso de gestión	https://nap.frt.utn.edu.ar
Red interna	172.16.90.0/24
Almacenamiento	local-lvm (LVM-Thin)

2.2. Esquema de direccionamiento IP

La dirección IP de cada contenedor se construye a partir del identificador (ID) asignado por Proxmox al momento de su creación, siguiendo el formato 172.16.90.ID. De esta manera, el último octeto coincide con el ID del contenedor.

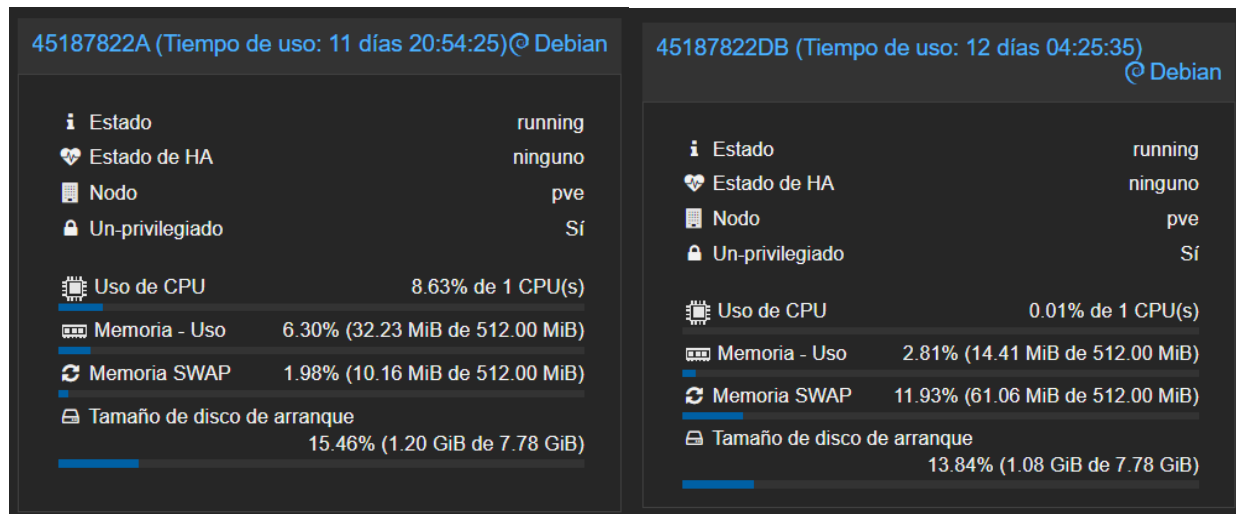
Parámetro	Contenedor Blog	Contenedor DB
ID asignado	202	203
Dirección IP	172.16.90.202	172.16.90.203
Máscara de subred	255.255.255.0 (/24)	255.255.255.0 (/24)
Puerta de enlace	172.16.90.1	172.16.90.1
DNS	172.16.90.1	172.16.90.1

3. Aprovisionamiento de los contenedores

Se crearon dos contenedores LXC con la plantilla Debian 12 standard, respetando estrictamente los recursos solicitados. La nomenclatura utiliza el DNI del alumno como base, agregándole el identificador correspondiente según la consigna.

Recurso	Contenedor Blog	Contenedor DB
Hostname	45187822A	45187822DB
Sistema operativo	Debian 12	Debian 12
Memoria	128 MB	128 MB
Swap	512 MB	512 MB
Procesadores	1 núcleo	1 núcleo
Almacenamiento	8 GB	8 GB
Tipo	No privilegiado	No privilegiado

Nota: se asignó una memoria de intercambio (swap) de 512 MB a cada contenedor para asegurar la estabilidad del sistema sin alterar el límite de 128 MB de memoria RAM solicitado.



Iniciar al arranque	No
Orden de inicio/apagado	order=any
Tipo de SO	debian
Arquitectura	amd64
/dev/console	Activado
Número TTY	2
Modo de consola	tty
Protección	No
Contenedores sin privilegios	Sí
Funcionalidades	nesting=1

4. Configuración de la capa de datos (Contenedor DB)

El contenedor de base de datos (172.16.90.203) ejecuta el motor MariaDB. Dada la restricción de 128 MB de memoria, se aplicó una configuración de bajo consumo para garantizar el funcionamiento estable del servicio.

4.1. Instalación del motor

```
apt update && apt upgrade -y
apt install -y mariadb-server
```

4.2. Ajuste de bajo consumo de memoria

Se creó el archivo `/etc/mysql/mariadb.conf.d/99-low-mem.cnf` con los siguientes parámetros, orientados a reducir el uso de memoria del motor:

```
[mysqld]
performance_schema = OFF
innodb_buffer_pool_size = 16M
key_buffer_size = 8M
```

```
max_connections = 20
bind-address = 0.0.0.0
```

4.3. Creación de la base, usuario y esquema

Se creó la base de datos blog y un usuario dedicado con acceso restringido exclusivamente a la dirección IP del contenedor de la aplicación, aplicando el principio de mínimo privilegio.

```
CREATE DATABASE blog CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'bloguser'@'172.16.90.202' IDENTIFIED BY '*****';
GRANT ALL ON blog.* TO 'bloguser'@'172.16.90.202';
FLUSH PRIVILEGES;
```

El esquema de la tabla que almacena las entradas del blog es el siguiente:

```
CREATE TABLE posts (
  id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  titulo VARCHAR(200) NOT NULL,
  contenido TEXT NOT NULL,
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id)
) ENGINE=InnoDB;
```

5. Configuración de la capa de aplicación (Contenedor Blog)

El contenedor de aplicación (172.16.90.202) aloja el blog desarrollado en Python con el framework Flask. La arquitectura sigue el patrón estándar de despliegue en producción: Nginx actúa como reverse proxy de cara al exterior y Gunicorn como servidor de aplicaciones que ejecuta el código Flask.

5.1. Arquitectura de la solución

El flujo de una solicitud HTTP recorre los siguientes componentes:

```
Navegador -> Nginx (:80) -> Gunicorn (127.0.0.1:8000) -> Flask -> MariaDB
(172.16.90.203)
```

Nginx además sirve directamente los archivos estáticos (la imagen personal y el informe en PDF), evitando sobrecargar al proceso de Python y optimizando el uso de recursos.

5.2. Componentes instalados

Servidor web	Nginx (reverse proxy)
Servidor de aplicaciones	Gunicorn (1 worker)
Framework	Flask (Python 3)
Conector a base de datos	PyMySQL
Gestión del servicio	systemd (blog.service)

5.3. Servicio systemd

Para asegurar que la aplicación se inicie automáticamente y se reinicie ante fallos, se definió una unidad de systemd que mantiene a Gunicorn en ejecución de forma permanente.

```
[Service]
WorkingDirectory=/app
ExecStart=/app/venv/bin/gunicorn --workers 1 --bind 127.0.0.1:8000 app:app
Restart=on-failure
```

5.4. Configuración de Nginx

```
server {
    listen 80;
    location /static/ {
        alias /app/static/;
    }
    location / {
        proxy_pass http://127.0.0.1:8000;
    }
}
```

6. Funcionalidad del blog

La página principal del blog presenta, en orden, los datos personales del alumno junto a su imagen, un formulario embebido para agregar nuevas entradas y el listado de entradas publicadas ordenadas de la más reciente a la más antigua. De esta forma se cumple el requisito de poder actualizar la página principal agregando un nuevo dato.

6.1. Características implementadas

- Datos personales e imagen del alumno en la cabecera del blog.
- Formulario embebido en la página principal para publicar nuevas entradas.
- Persistencia de las entradas en la base de datos del segundo contenedor.
- Listado dinámico de entradas ordenado por fecha descendente.
- Enlace al informe en formato PDF publicado en el propio blog.

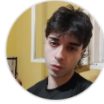
6.2. Consideraciones de seguridad

Las operaciones contra la base de datos se realizan mediante consultas parametrizadas, lo que previene ataques de inyección SQL. El usuario de base de datos posee permisos restringidos a una única base y solo acepta conexiones provenientes de la IP del contenedor de aplicación. La aplicación implementa el patrón Post/Redirect/Get para evitar el reenvío accidental de formularios.

BLOG PERSONAL

[INICIO](#) [INFORME PDF](#)

Entrada publicada correctamente.



Victor Manuel Miranda López

Edad 22 años
DNI 45 187 822
Carrera Ingeniería en Sistemas de Información
Facultad UTN FRT
Legajo 56353

[ABRIR INFORME PDF](#)

NUEVA ENTRADA

TÍTULO

CONTENIDO

[PUBLICAR](#)

ENTRADAS DEL BLOG

prueba 4

28 de junio de 2026, 12:43

prueba

prueba 3

27 de junio de 2026, 15:09

prueba

prueba 2

27 de junio de 2026, 15:06

prueba

prueba 1

27 de junio de 2026, 15:05

prueba

7. Publicación del blog bajo la URL institucional

La cátedra asignó a la implementación una URL pública bajo un prefijo de ruta derivado del DNI del alumno. El acceso al blog desde el exterior se realiza a través de un servidor proxy administrado por la cátedra, que reenvía las solicitudes recibidas en dicho prefijo hacia el contenedor de aplicación.

URL asignada	http://nap.frt.utn.edu.ar/45187822/
Prefijo de ruta	/45187822/
Proxy externo	Administrado por la cátedra (NAP)
Destino interno	172.16.90.202:80 (Nginx del contenedor)

La incorporación de este prefijo requirió adaptar tanto la configuración del servidor web interno como la propia aplicación, dado que originalmente el blog estaba pensado para responder en la raíz del sitio.

7.1. Adaptación del servidor web interno (Nginx)

Se ajustó la configuración de Nginx del contenedor de aplicación para reconocer el prefijo asignado y redirigir el tráfico hacia Unicorn, sirviendo además los recursos estáticos bajo la misma ruta base.

```
server {
    listen 80;
    server_name _;

    location /45187822/static/ {
        alias /app/static/;
    }

    location /45187822/ {
        proxy_pass http://127.0.0.1:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Script-Name /45187822/;
    }
}
```

7.2. Adaptación de la aplicación (Flask)

Se configuró la aplicación para que reconociera la ruta base asignada, definiendo el prefijo de los recursos estáticos y la raíz de la aplicación:

```
app = Flask(__name__, static_url_path='/45187822/static')
app.config['APPLICATION_ROOT'] = '/45187822/'
```

7.3. Dificultad encontrada: entrega de recursos estáticos

Durante las pruebas de acceso externo se detectó que, si bien la página principal del blog se mostraba correctamente, los recursos estáticos (hoja de estilos, imagen personal e informe en

PDF) no se cargaban. Al solicitarlos directamente, el proxy externo respondía con un error HTTP 501.

Se determinó que el proxy de la cátedra reenvía hacia el contenedor únicamente las solicitudes dirigidas exactamente al prefijo /45187822/, pero no las rutas anidadas correspondientes a los archivos estáticos (/45187822/static/...). En consecuencia, dichos recursos quedaban inaccesibles desde el exterior, a pesar de que el Nginx interno del contenedor los servía correctamente (verificado con una respuesta HTTP 200 desde dentro de la red).

```
# Desde dentro del contenedor: el recurso se sirve bien
curl -I http://127.0.0.1/45187822/static/style.css -> HTTP/1.1 200 OK

# Desde el exterior, a través del proxy de la cátedra
http://nap.frt.utn.edu.ar/45187822/static/style.css -> HTTP 501
```

7.4. Solución adoptada

Dado que la configuración del proxy externo es administrada por la cátedra y no puede modificarse desde el contenedor, se optó por eliminar la dependencia de rutas estáticas anidadas, incorporando los recursos directamente en el contenido entregado por la aplicación:

- Hoja de estilos: el contenido de la CSS se incorporó de forma embebida en el documento HTML, dentro de una etiqueta de estilo, evitando la solicitud a un archivo externo.
- Imagen personal: se codificó en formato Base64 y se embebió directamente en el atributo de origen de la imagen, de modo que viaja dentro del propio HTML.
- Informe en PDF y envío del formulario: se reescribieron sus enlaces y la acción del formulario utilizando la ruta absoluta con el prefijo /45187822/, en lugar de rutas relativas a la raíz.
- Redirecciones: se ajustó la redirección posterior a la publicación de una entrada para que apunte a la ruta absoluta del blog, evitando que el navegador fuera enviado a la raíz del dominio.

Con estas adaptaciones, la totalidad de la interfaz del blog -estilos, imagen personal, descarga del informe y publicación de nuevas entradas- quedó operativa a través de la URL institucional asignada, sin depender del reenvío de rutas estáticas por parte del proxy externo.

8. Pruebas y verificación

Se verificó el correcto funcionamiento de la solución mediante las siguientes pruebas:

8.1. Conectividad de la aplicación

Desde el contenedor de aplicación se comprobó que el servidor responde correctamente, devolviendo el HTML del blog:

```
curl http://127.0.0.1:8000
```

8.2. Comunicación entre contenedores

Se validó la conectividad de la capa web hacia la capa de datos, confirmando que la aplicación accede a la base de datos alojada en el segundo contenedor.

8.3. Prueba funcional

Se cargó una nueva entrada a través del formulario de la página principal, verificando que se almacena en la base de datos y se visualiza inmediatamente en el listado del blog.

```
root@45187822DB:~# mysql -u root blog -e "SELECT * FROM posts ORDER BY fecha_creacion DESC;"
+-----+-----+-----+-----+
| id | titulo | contenido | fecha_creacion |
+-----+-----+-----+-----+
| 3 | prueba 3 | prueba | 2026-06-27 18:09:02 |
| 2 | prueba 2 | prueba | 2026-06-27 18:06:15 |
| 1 | prueba 1 | prueba | 2026-06-27 18:05:33 |
+-----+-----+-----+-----+
```

9. Conclusiones

El trabajo permitió implementar de forma completa un servicio de blog personal sobre una infraestructura virtualizada, cumpliendo con todas las especificaciones técnicas requeridas: recursos acotados, direccionamiento IP estático derivado del ID de cada contenedor, nomenclatura basada en el DNI y publicación del informe en la propia implementación.

La separación en dos contenedores -capa de aplicación y capa de datos- refleja una arquitectura realista de consolidación de servidores, donde múltiples servicios independientes se ejecutan de manera aislada sobre un mismo hardware físico, aprovechando la eficiencia de la virtualización a nivel de sistema operativo. El uso de contenedores LXC, con su bajo consumo de recursos, resultó especialmente adecuado para el entorno restringido de 128 MB de memoria por contenedor.

Asimismo, la publicación del servicio bajo la URL institucional asignada exigió comprender el comportamiento del proxy inverso administrado por la cátedra y adaptar la aplicación a sus restricciones. La resolución de la entrega de recursos estáticos -embebiendo los estilos y la imagen dentro del propio contenido y empleando rutas absolutas- constituyó un ejercicio práctico valioso sobre el despliegue de aplicaciones web detrás de un proxy con prefijo de ruta, una situación habitual en entornos productivos reales.

Anexo: stack tecnológico

Capa	Tecnología	Contenedor
Servidor web / Proxy	Nginx	Blog (202)
Servidor de aplicaciones	Gunicorn	Blog (202)
Lenguaje / Framework	Python 3 / Flask	Blog (202)
Conector de base de datos	PyMySQL	Blog (202)
Motor de base de datos	MariaDB	DB (203)
Sistema operativo	Debian 12	Ambos
Plataforma de virtualización	Proxmox VE 8.2.2	Nodo pve